

3.4.1 Algoritmi di Labeling (Etichettatura di Regioni)

Gli **algoritmi di labeling** (o **label region**) sono essenziali per risolvere il limite della **segmentazione a soglia**, che non è in grado di distinguere oggetti topologicamente diversi ai quali è associato lo stesso livello di segnale.

3.4.1.1 Scopo e Necessità

Quando si applica una segmentazione a soglia a un'immagine contenente regioni separate ma con intensità simili, il risultato è spesso un'unica **maschera binaria** che non distingue le singole componenti connesse.

L'algoritmo di labeling scompone questa maschera binaria in base alla **connettività topologica**, assegnando un'etichetta (label) unica a ciascun oggetto distinto.

3.4.1.2 Funzionamento dell'Algoritmo di Labeling

L'algoritmo opera su un'**immagine binaria** e identifica i **componenti connessi** (*blob* o *regioni connesse*).

Procedura:

1. **Inizio:** Si seleziona il primo pixel dell'immagine (tipicamente procedendo in *raster scan*). Se il pixel non è ancora stato assegnato a un *blob* (e ha valore 1 nella maschera), si **crea un nuovo gruppo** (blob).
2. **Espansione del Blob:** Per il pixel corrente, si esamina il suo **intorno** (kernel) e si aggiungono i pixel connessi.
3. **Iterazione:** Completato il primo *blob*, si prende il pixel successivo non ancora assegnato e si crea un **nuovo blob** con un'etichetta diversa.
4. **Termine:** Il processo si itera finché tutti i pixel sono stati assegnati.

3.4.1.2.1 Definizione dell'Intorno

L'intorno può includere:

- Connettività a **4 vicini** (senza diagonali)
- Connettività a **8 vicini** (incluse le diagonali)

3.4.1.3 Elaborazione Post-Labeling

I *blob* ottenuti possono essere classificati per **grandezza** o eliminati se troppo piccoli (considerati **rumore**).

In **MATLAB**: funzioni `bwlabel` o `bwconncomp`.

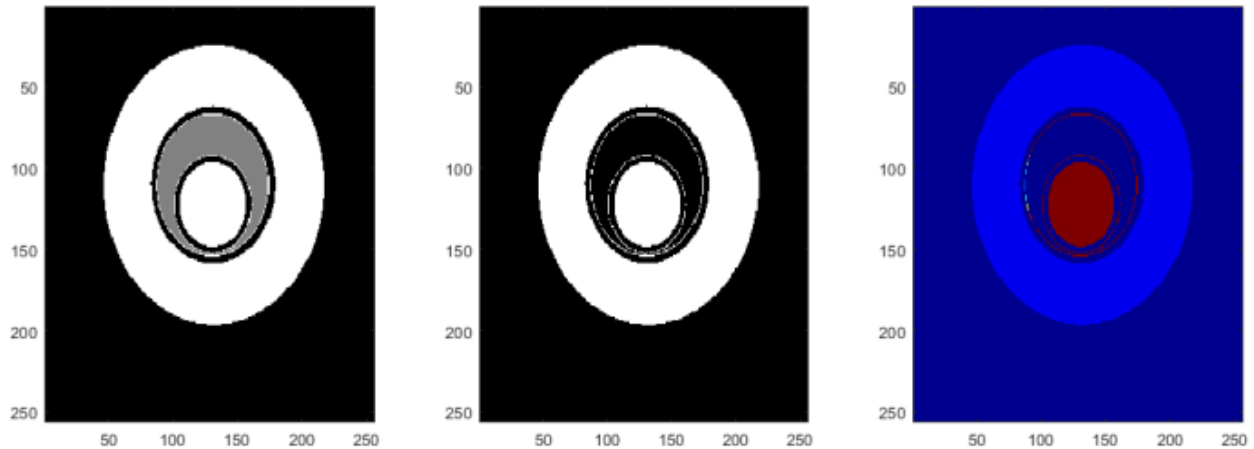


Figura 1: Pasted image 20251129203809.png

3.4.2 Region Growing (Crescita di Regione)

Il **Region Growing** è un'estensione della segmentazione a soglia che sfrutta le **informazioni spaziali** per estendere la segmentazione da un punto iniziale (seme) a tutta la regione di interesse connessa.

3.4.2.1 Procedura

1. **Scelta del Seed Pixel:** Si definisce un punto di partenza (**'seed pixel'**) all'interno del pattern da segmentare.
2. **Crescita della Regione:** A partire dal seme, la regione **'cresce'** ricorsivamente aggiungendo i pixel adiacenti che soddisfano un **criterio di somiglianza** prefissato.
3. **Arresto:** Quando la crescita della regione si ferma, il processo termina.

3.4.2.2 Criteri di Similarità e Soglia (T)

3.4.2.2.1 Immagini Binarie

Nel caso di immagini binarie, il criterio di somiglianza è l'**uguaglianza del valore dei pixel**. In questo caso, l'algoritmo si comporta come l'**algoritmo di Labeling**.

3.4.2.2.2 Immagini a Livello di Grigio

Si include un pixel se il suo valore differisce da quello di un pixel adiacente per meno di una soglia T :

$$ABS(s_0 - s_1) < T$$

3.4.2.2.3 Definizione di T

- **Valore Assoluto:** T definito come numero di livelli di grigio (es. $T = 50$)
- **Percentuale Relativa:** $\frac{ABS(s_0 - s_1)}{MEAN(s_0, s_1)} < T$
- **Proporzionale al Rumore:** $T = 1.96\sigma$ per includere il 95% dei pixel della regione

In MATLAB: funzione grayconnected.

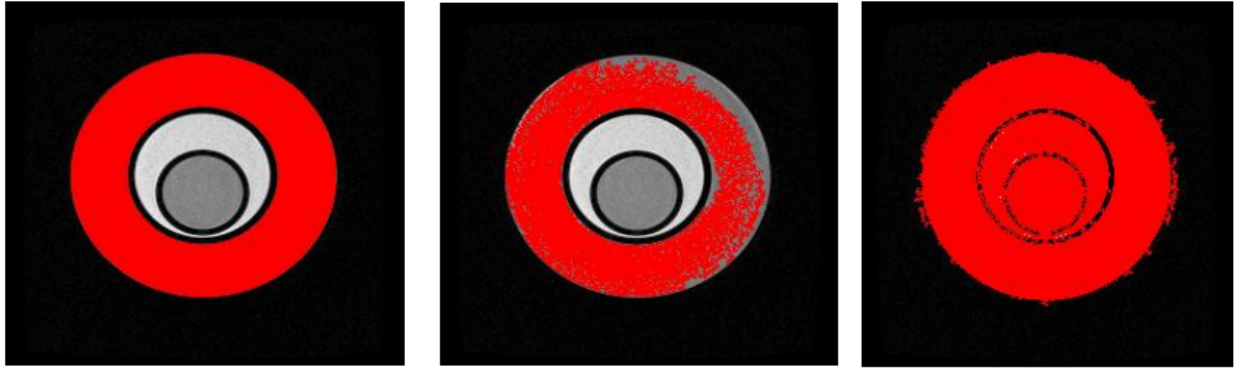


Figura 2: Pasted image 20251129203837.png

3.4.3 Segmentazione a Contorni

Un approccio diverso consiste nell'identificare sull'immagine i **contorni dei pattern**, che corrispondono alle zone di transizione tra due tessuti diversi.

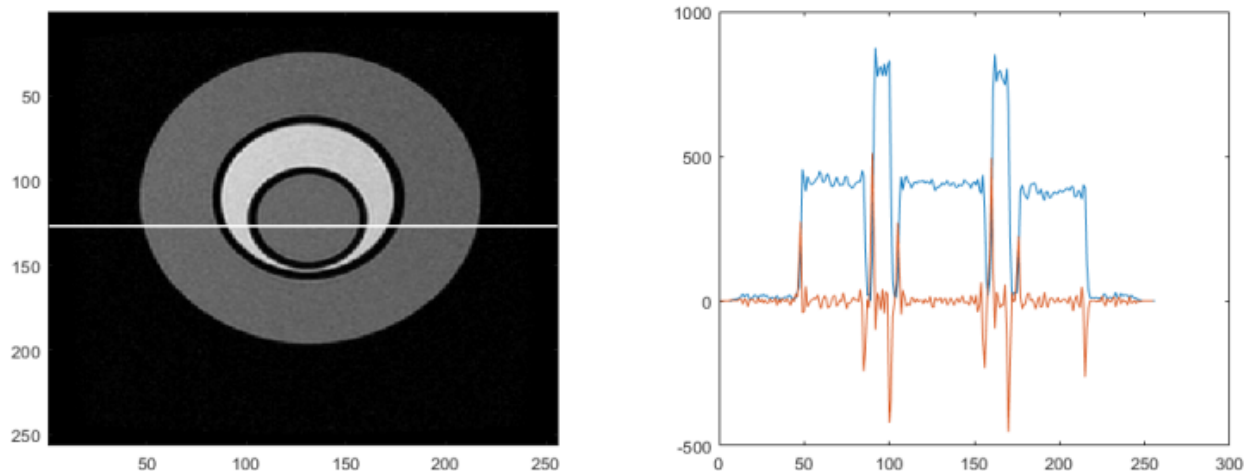


Figura 3: Pasted image 20251129203925.png

I contorni sono caratterizzati da una **transizione netta del segnale**. Calcolando la derivata del profilo, valori alti della derivata corrispondono ai contorni.

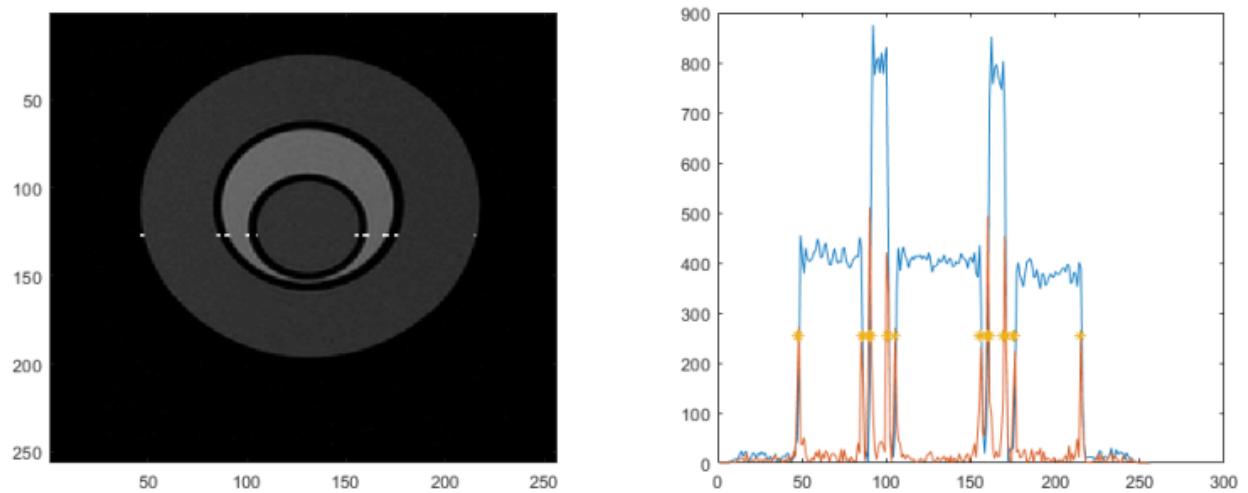


Figura 4: Pasted image 20251129203940.png

3.4.3.1 Rilevamento dei Contorni

Una **mappa di gradiente** dell'immagine consente di identificare i contorni. In MATLAB la funzione `edge` trova i contorni attraverso filtri come Sobel, Roberts, Laplaciano.

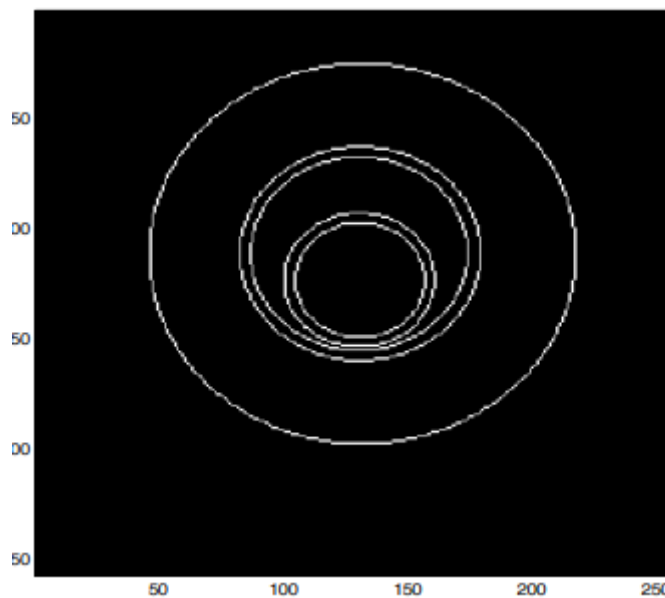


Figura 5: Pasted image 20251129204019.png

3.4.3.2 Collegamento dei Contorni (Algoritmo di Canny)

La creazione di un contorno continuo utilizza tipicamente due parametri:

1. **Intensità della mappa di gradiente:** $||\mathbf{G}(x, y) - \mathbf{G}(x', y')|| \leq T$

2. **Direzione del gradiente:** $|\alpha(x, y) - \alpha(x', y')| \leq A$

dove:

$$\alpha(x, y) = \tan^{-1} \left\{ \frac{\partial f(x, y) / \partial y}{\partial f(x, y) / \partial x} \right\}$$

3.4.4 Active Contours (Snakes)

I metodi basati su soglia e clustering non incorporano un **modello geometrico o topologico** dell'oggetto. Gli **Active Contours (Snakes)** introducono informazioni *a priori* sulla forma dell'oggetto.

L'algoritmo (Kass) inizializza una curva (**snake**) e la deforma per farla convergere su un contorno che corrisponde a un **minimo locale di energia**.

3.4.4.1 Definizione e Parametrizzazione

La curva C è definita come un insieme di N punti ordinati, o come curva parametrica:

$$C(s) = \{(x(s), y(s)) : 0 \leq s \leq 1\}$$

3.4.4.2 Energia dello Snake

L'energia totale è la somma di una componente **interna** (E_{int}) e una **esterna** (E_{imm}):

$$E(C) = \int_0^1 E_{int} + E_{imm} ds$$

3.4.4.2.1 Energia Interna

$$E_{int}(C(s)) = \alpha |C'(s)|^2 + \beta |C''(s)|^2$$

- $\alpha |C'(s)|^2$ (Energia Elastica): Tendenza a mantenere la lunghezza
- $\beta |C''(s)|^2$ (Energia di Curvatura): Tendenza a opporsi alle modifiche repentine della curvatura

3.4.4.2.2 Energia Esterna

Basata sul gradiente:

$$E_{imm}(C(s)) = -|\nabla I(x, y)|^2$$

O con filtro Gaussiano:

$$E_{imm}(C(s)) = -|\nabla(G_\sigma(x, y) * I(x, y))|^2$$

3.4.4.3 Modelli Region-Based (Chan-Vese)

L'algoritmo di **Chan-Vese** valuta l'omogeneità della regione segmentata:

$$F(C) = \int_{inside(C)} |I - C_1|^2 ds + \int_{outside(C)} |I - C_2|^2 ds$$

In MATLAB: funzione `activecontour`.

3.4.5 Level Set (Contorno Attivo Implicito)

L'algoritmo **Level Set** risolve i problemi topologici degli Snakes definendo il contorno in modo **implicito** come lo **zero-level set** di una funzione $\phi(x, y)$:

$$C = \{(x, y) : \phi(x, y) = 0\}$$

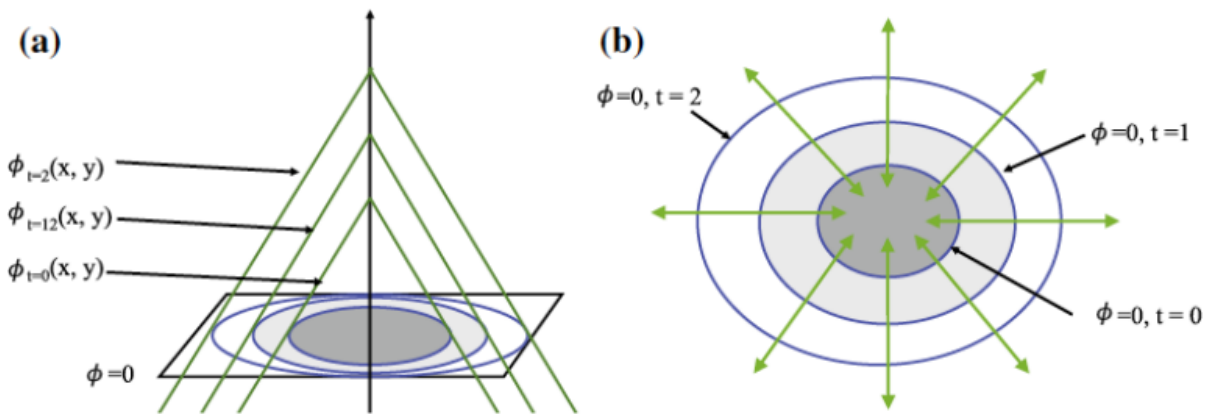


Figura 6: Pasted image 20251129204248.png

3.4.5.1 Equazione di Evoluzione

$$\frac{\partial \phi}{\partial t} + F|\nabla \phi| = 0$$

dove F è la “**speed function**”:

$$F(x, y) = \frac{1}{1 + \lambda |\nabla I(x, y)|}$$

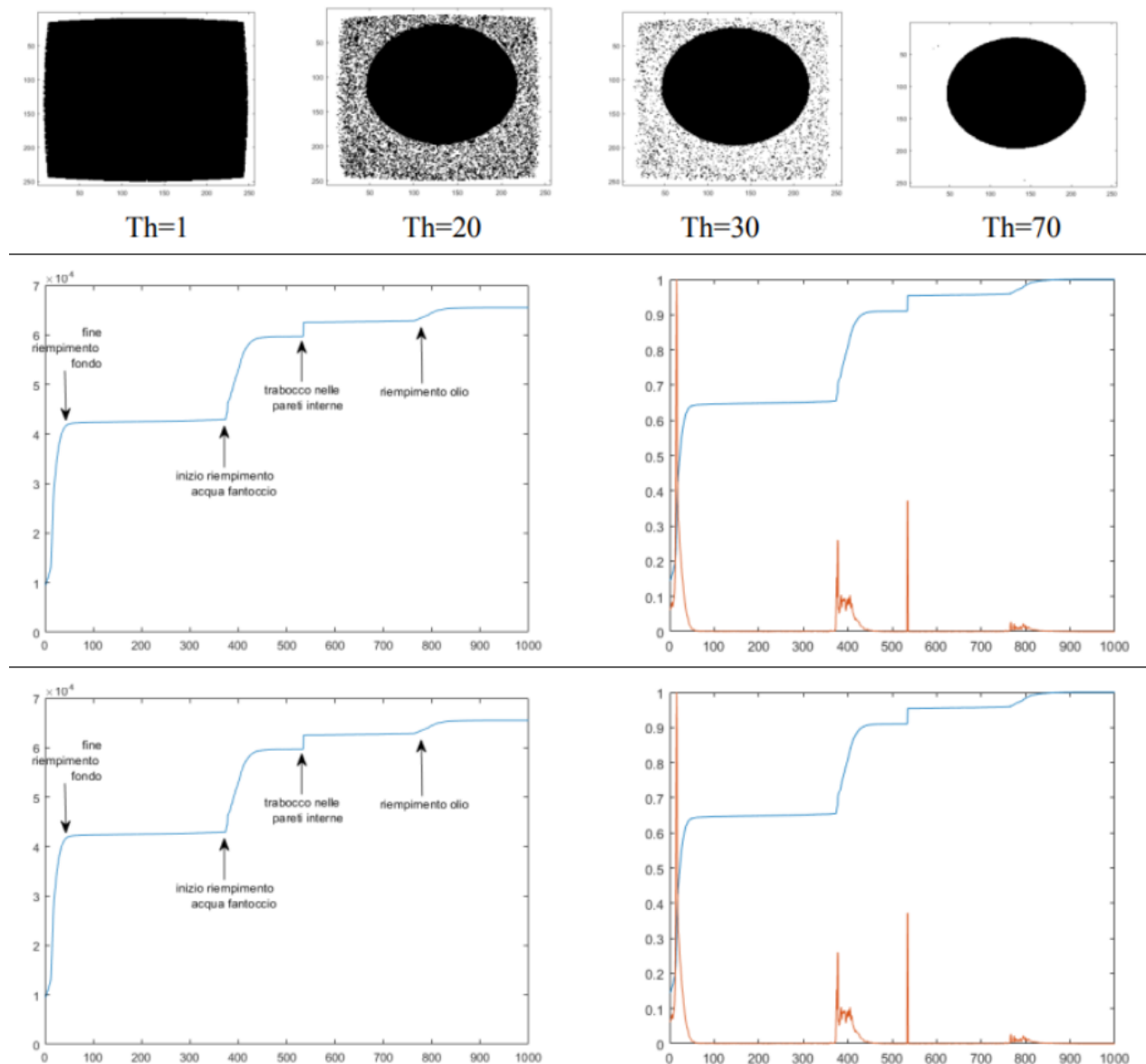
3.4.5.2 Vantaggio Topologico

Il contorno definito da ϕ può **dividersi in due o più contorni** (o fondersi) durante la progressione, offrendo maggiore **flessibilità topologica**.

3.4.6 Watershed Transform

La **watershed transform** (spartiacque) vede il livello di grigio come la profondità di un pixel. L'immagine viene "riempita con acqua" e quando si arriva ad un watershed, l'acqua trabocca in una regione vicina.

Consideriamo il fantoccio MR in figura e il relativo profilo. L'asse Y del profilo esprime in questo caso la distanza dal fondo che ha riferimento 0. Immettiamo l'acqua da un seed posto sul fondo. L'acqua riempirà i pixel connessi al seed e più "bassi" (quindi con livello di grigio minore) del seed. Questa funzione è implementata in Matlab da `grayconnected`. Man mano che sale l'acqua i pixel del fondo vengono sommersi e la maschera risultante cresce.



Se tracciamo il grafico dell'area della maschera in funzione del livello otteniamo la situazione in figura, dove come si è detto periodi di area stabile indicano che sono state riconosciute le strutture principali del fantoccio. L'algoritmo watershed attraverso soglie opportune riconosce le transizioni tra i periodi di riempimento e produce le relative maschere. Un metodo semplice è calcolare il gradiente della curva (in rosso in figura) e scegliere le soglie tra due picchi del gradiente.

Tracciando il **grafico dell'area della maschera in funzione del livello**, i **periodi di area stabile** indicano le strutture principali.

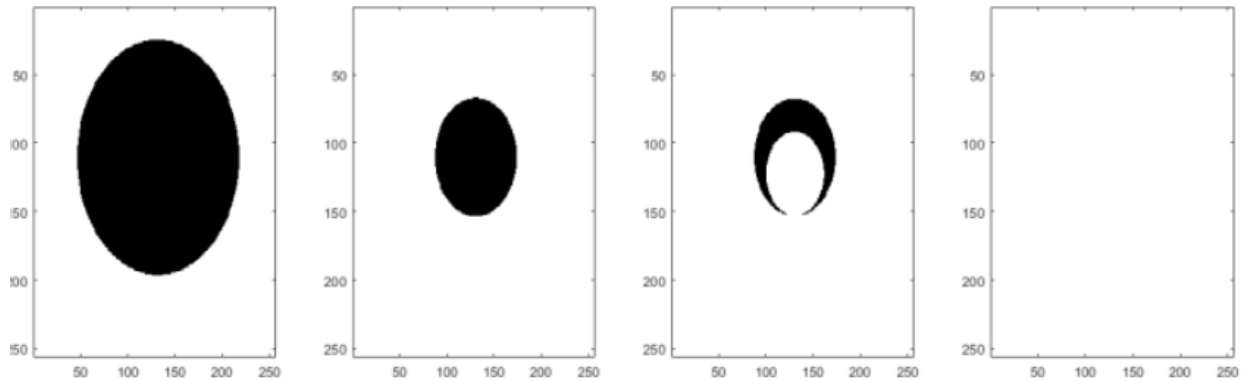


Figura 7: Pasted image 20260105161027.png

Mappe per $th = 200, th = 500, th = 650, th = 1900$.

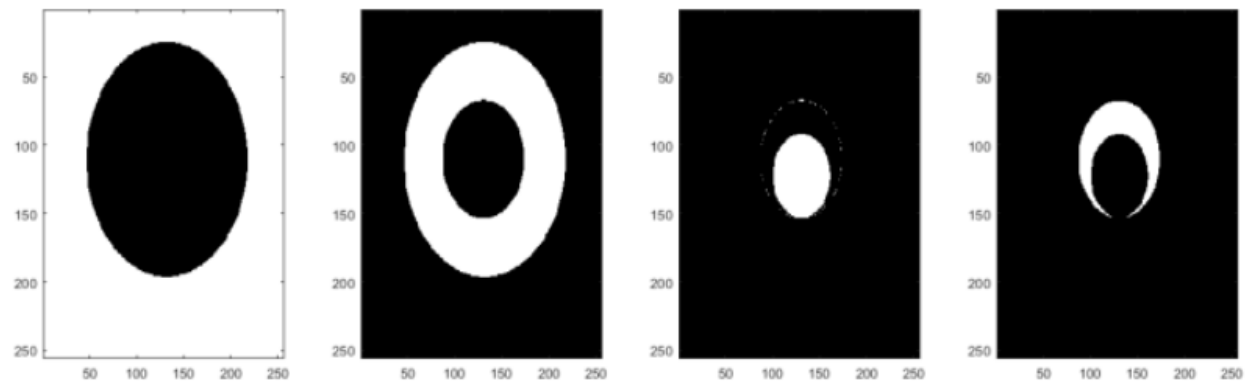


Figura 8: Pasted image 20260105161037.png

Mappe ottenute per sottrazione progressiva delle maschere. Notiamo che pur utilizzando un approccio a soglia, l'algoritmo riesce a distinguere regioni topologicamente non connesse.

3.4.7 Algoritmo MSERs

L'algoritmo **Maximally Stable Extremal Regions (MSERs)** ricerca le regioni estreme massimamente stabili. Le regioni estreme sono definite come le regioni connesse i cui livelli di grigio sono tutti al di sopra o al di sotto dei valori dei pixel che circondano la regione.

In MATLAB: funzione `detectMSERFeatures`.

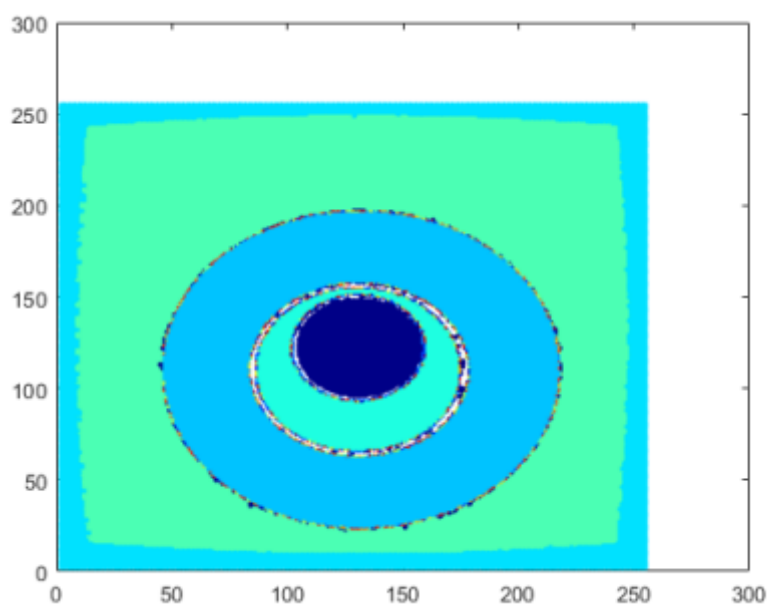
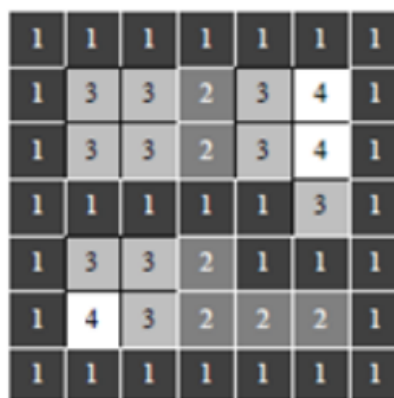
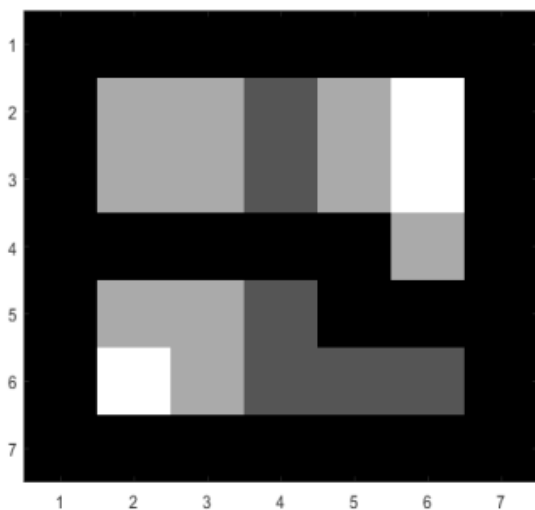


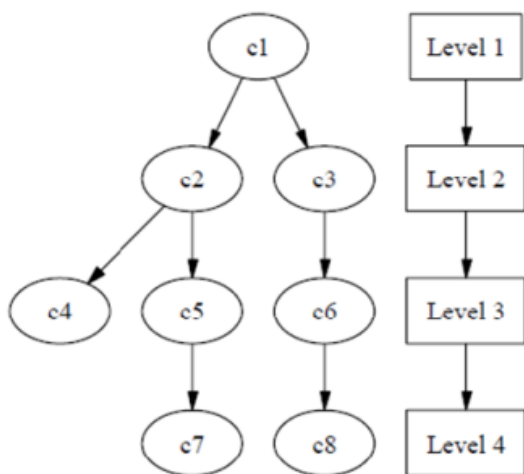
Figura 9: Pasted image 20251129204429.png

3.4.8 Component Tree

L'approccio **Component Tree** permette di memorizzare il contenuto di un'immagine in un **albero**, permettendo di realizzare algoritmi di elaborazione in modo efficiente.



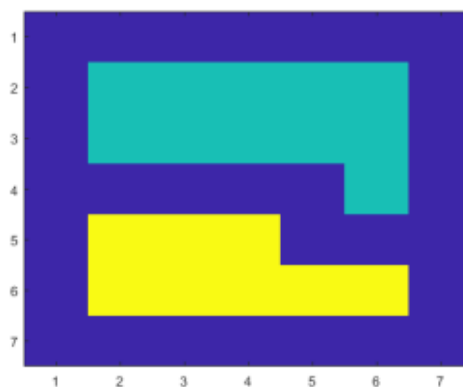
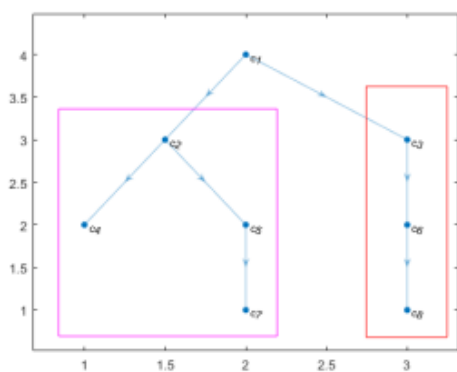
F



(a)

c1	c1	c1	c1	c1	c1	c1
c1	c4	c4	c2	c5	c7	c1
c1	c4	c4	c2	c5	c7	c1
c1	c1	c1	c1	c1	c5	c1
c1	c6	c6	c3	c1	c1	c1
c1	c8	c6	c3	c3	c3	c1
c1	c1	c1	c1	c1	c1	c1

(b)



I livelli dell'albero sono pari al numero di livelli di grigio presenti nell'immagine e il numero di nodi è uguale al numero di pattern presenti.

3.4.9 Skeletonization

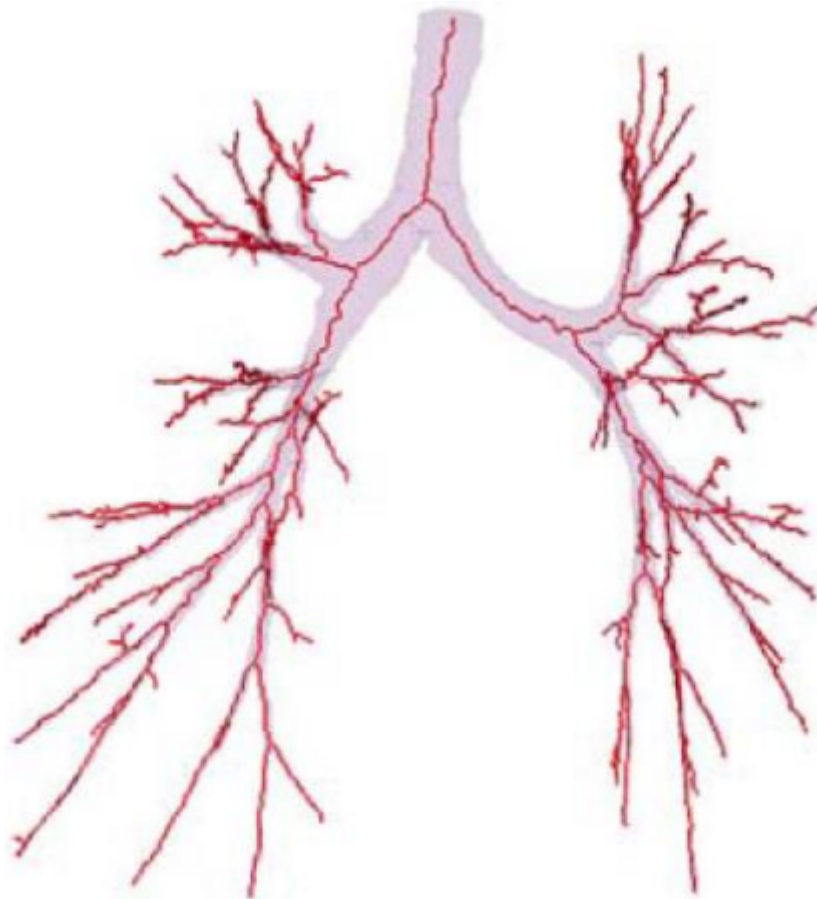


Figura 10: Pasted image 20251129204552.png

Lo **scheletro topologico** di una maschera si ottiene “assottigliando” la maschera stessa fino ad ottenerne una versione essenziale, tipicamente composta da linee.

3.4.9.1 Approcci

1. **Thinning Morfologico:** Basato sulla **hit-and-miss transform** (HAM)

L'operazione di thinning si ottiene calcolando:

$$M_{thin} = M - M_{HAM}$$

2. **Distance Transform:** Per ogni punto della maschera viene calcolata la distanza minima dal contorno

In MATLAB:

- Skeletonization: `bwmorph` con opzione `'skel'`

	1	
0	1	1
0	0	

	1	
1	1	0
	0	0

	0	0
1	1	0
	1	

0	0	
0	1	1
	1	

Figura 11: Pasted image 20251129204542.png

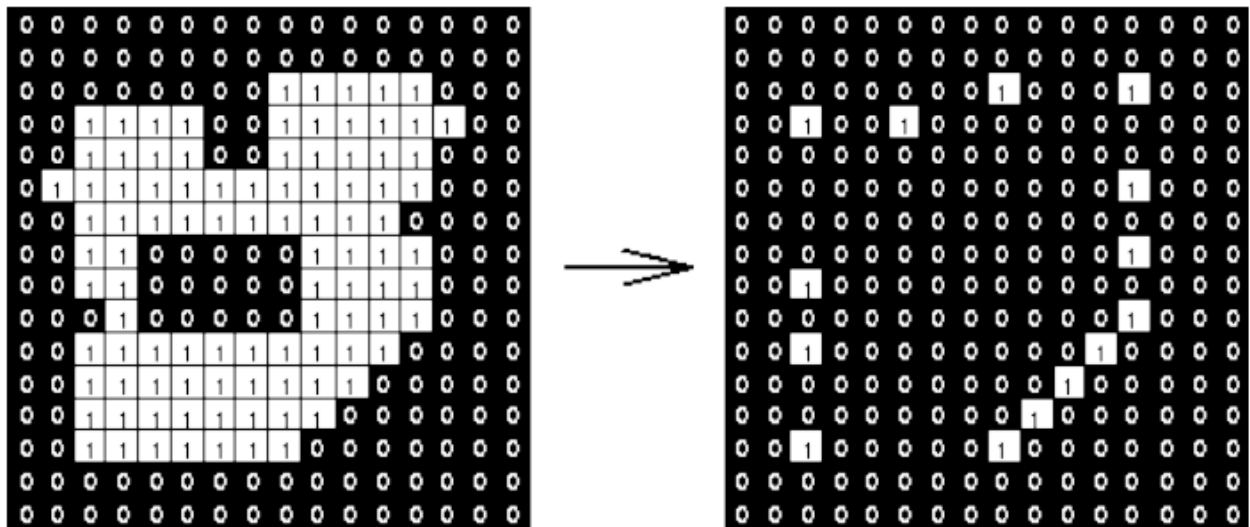


Figura 12: Pasted image 20251129204614.png

$$\begin{bmatrix} 0 & 0 & 0 \\ & 1 & \\ 1 & 1 & 1 \end{bmatrix} \quad \begin{bmatrix} & 0 & 0 \\ 1 & 1 & 0 \\ & 1 & \end{bmatrix}$$

Figura 13: Pasted image 20251129204646.png

- Distance transform: `bwdist`
- Hit-and-miss: `bwhitmiss`

NOTA: I metodi classici descritti in questa sezione possono essere confrontati con gli approcci basati su **Deep Learning**: - U-Net per Segmentazione Semantica - architettura encoder-decoder con skip connections - Reti Neurali Convoluzionali (CNN) - fondamenti delle CNN per image analysis

Le CNN superano i metodi classici quando sono disponibili dataset annotati sufficientemente grandi.

← Clustering | □ | Validazione →